

# Controlling the Donkey game using reinforcement learning

1<sup>st</sup> Daniil Kucherenko

Department of Computer Science and Information Technology  
Technical University of Kosice  
Kosice, Slovakia  
daniil.kucherenko@student.tuke.sk

2<sup>nd</sup> Valeriia Buhaiova

Department of Computer Science and Information Technology  
Technical University of Kosice  
Kosice, Slovakia  
valeriia.buhaiova@student.tuke.sk

**Abstract**—This project focuses on training a reinforcement learning agent to play the classic MS-DOS game *Donkey*, also known as *DONKEY.BAS* [1]. The task is formulated as a sequential decision-making problem in which the agent learns to select actions that avoid obstacles and maximize the achieved score.

The implemented solution compares three reinforcement learning methods: Q-learning [12], Double Q-learning [10], and an Actor-Critic approach [3]. Neural networks are used as function approximators for value functions and policies, following the general principles of deep reinforcement learning applied to game environments [6].

The algorithms are evaluated using repeated Monte Carlo experiments, where multiple independent training runs are averaged to obtain more reliable performance estimates [4]. The comparison focuses on training time, average achieved score, score variance, and the influence of neural network depth on the final performance.

## I. INTRODUCTION

Reinforcement learning is a machine learning approach in which an agent learns to make decisions through interaction with an environment. At each step, the agent observes the current state, selects an action, receives a reward, and updates its behaviour based on the collected experience. This makes reinforcement learning suitable for game-playing tasks, where the objective can be defined by the achieved score, survival time, or successful avoidance of obstacles.

This project focuses on training an autonomous agent to play the classic MS-DOS game *Donkey*, also known as *DONKEY.BAS* [1]. The game represents a simple driving scenario in which the player controls a car and avoids obstacles on the road. Despite its simple mechanics, it provides a useful environment for studying and comparing reinforcement learning algorithms.

The goal of this work is to implement and compare three reinforcement learning approaches: Q-learning [12], Double Q-learning [10], and an Actor-Critic method [3]. Neural networks are used as function approximators for value functions

or policies, following the general principles of deep reinforcement learning [6]. The implemented methods are evaluated with respect to training time, average achieved score, score variance, and the influence of neural network depth.

## II. BACKGROUND

### A. Donkey Game

*DONKEY.BAS* is a simple driving game in which the player controls a car and must avoid donkeys appearing on the road. The game was originally written in BASIC and is one of the early examples of a small interactive computer game distributed with IBM PC systems [1]. In this work, the game is used as an environment for training reinforcement learning agents. The goal of the agent is to survive as long as possible by selecting the correct action at each step.

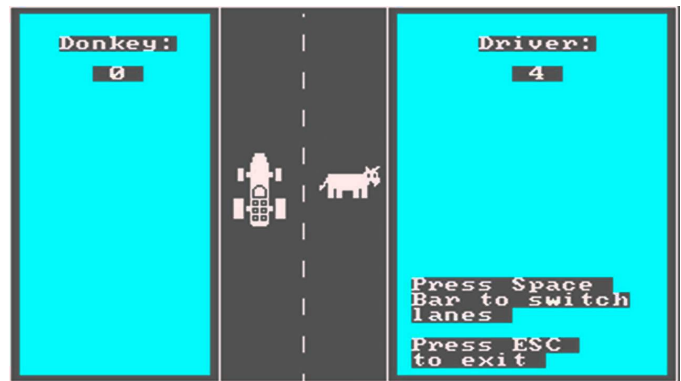


Fig. 1. Example screen from the Donkey game used as the reinforcement learning environment.

### B. Reinforcement Learning

Reinforcement learning is a machine learning approach in which an agent learns to make decisions through interaction with an environment. At each time step, the agent observes a state  $s_t$ , selects an action  $a_t$ , receives a reward  $r_t$ , and moves

to a new state  $s_{t+1}$ . The objective is to learn a policy that maximizes the expected cumulative reward over time [9].

In the Donkey environment, the state describes the current game situation, the action determines whether the car jumps or continues moving normally, and the reward evaluates the result of this decision. A positive reward is given for successful progress, while a negative reward is given for crashes or incorrect actions.

### C. Q-learning

Q-learning is a model-free, off-policy reinforcement learning algorithm that learns the action-value function  $Q(s, a)$ , representing the expected cumulative reward for taking action  $a$  in state  $s$  and following the optimal policy thereafter. The Q-value update rule is defined as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

where  $\alpha$  is the learning rate,  $\gamma$  is the discount factor, and  $r_t$  is the received reward. The key idea is that the agent does not need a model of the environment — it learns purely from interactions by iteratively improving its estimates of Q-values. Action selection is typically governed by an  $\varepsilon$ -greedy policy, where the agent explores randomly with probability  $\varepsilon$  and exploits the current best action otherwise.

### D. Double Q-learning

A known limitation of standard Q-learning is the systematic overestimation of Q-values. This occurs because the same network is used both to select the best action and to evaluate it, which introduces a positive bias in the target values [11].

Double Q-learning addresses this by decoupling action selection from action evaluation. The action with the highest Q-value is selected using one network, while a separate network evaluates that action:

$$y_t = r_t + \gamma Q_{\theta} \left( s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a') \right).$$

This reduces overestimation bias and generally leads to more stable training and more accurate value estimates compared to standard Q-learning.

### E. Actor-Critic Method

Actor-Critic methods combine two main reinforcement learning approaches: policy-based learning and value-based learning. The actor represents a parameterized policy  $\pi_{\theta}(a|s)$ , which defines the probability of selecting an action  $a$  in a given state  $s$ . The critic estimates a value function and evaluates the quality of states under the current policy. In this work, the critic estimates the state-value function  $V_w(s)$ , where  $w$  denotes the critic parameters. The actor is responsible for action selection, while the critic provides feedback used to improve the policy [3], [9].

The general interaction between the actor, critic, and environment is shown in Figure 2. The agent observes the current state  $s_t$ , the actor selects an action  $a_t$  according to the current policy, and the environment returns a reward  $r_t$  and a new state

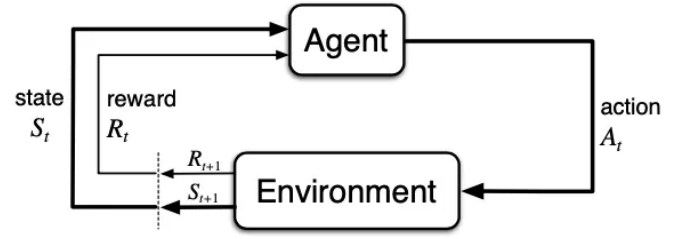


Fig. 2. General structure of the Actor-Critic method. The actor selects actions according to a policy, while the critic evaluates states and provides feedback used to improve the policy [14].

$s_{t+1}$ . The critic evaluates the current state and produces the value estimate  $V_w(s_t)$ . This estimate is then compared with a target value in order to determine whether the selected action led to a better or worse result than expected.

A common way to update the actor is to use the policy-gradient objective. The goal is to maximize the expected return:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}[G_t],$$

where  $G_t$  is the return obtained by following policy  $\pi_{\theta}$ . The policy-gradient update can be written as:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t|s_t) A_t],$$

where  $A_t$  is the advantage estimate [9].

The advantage function measures how much better an action is compared to the expected value of the current state:

$$A_t = Q(s_t, a_t) - V(s_t).$$

If the advantage is positive, the selected action performed better than expected and its probability should be increased. If the advantage is negative, the selected action performed worse than expected and its probability should be decreased.

In the implemented one-step Actor-Critic method, the critic target is computed from the immediate reward and the estimated value of the next state:

$$y_t = \begin{cases} r_t, & \text{if } s_{t+1} \text{ is terminal,} \\ r_t + \gamma V_w(s_{t+1}), & \text{otherwise.} \end{cases}$$

Here,  $\gamma$  is the discount factor. The temporal-difference error is then computed as:

$$\delta_t = y_t - V_w(s_t).$$

This temporal-difference error is used as a one-step estimate of the advantage:

$$A_t \approx \delta_t.$$

The critic is trained by minimizing the squared temporal-difference error:

$$L_{\text{critic}} = \frac{1}{2} (y_t - V_w(s_t))^2.$$

The actor is trained using the policy-gradient loss:

$$L_{\text{actor}} = -\log \pi_{\theta}(a_t|s_t) \delta_t.$$

Since gradient-based optimization frameworks usually minimize a loss function, the negative sign is used in the actor loss. This means that actions with a positive temporal-difference error become more likely, while actions with a negative temporal-difference error become less likely.

An optional entropy term can also be added to the actor loss in order to support exploration:

$$L_{\text{actor}} = -\log \pi_{\theta}(a_t|s_t)\delta_t - \beta H(\pi_{\theta}(\cdot|s_t)),$$

where  $H(\pi_{\theta}(\cdot|s_t))$  is the entropy of the policy distribution and  $\beta$  controls the strength of the entropy bonus. This term encourages the policy to avoid becoming deterministic too early during training.

The one-step variant updates the actor and critic after every transition. This makes the method simple and computationally efficient, but the resulting advantage estimate can be noisy because it uses only one immediate reward and one bootstrap estimate. More general Actor-Critic methods can use multi-step returns. In an  $n$ -step Actor-Critic method, the target is computed from several future rewards before bootstrapping from the value function:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_w(s_{t+n}).$$

This approach represents a compromise between one-step temporal-difference learning and Monte Carlo learning. A larger value of  $n$  uses more real rewards from the environment, but it also delays the update and may increase variance. Multi-step actor-critic updates are used, for example, in asynchronous advantage actor-critic methods [5], [9].

In the Monte Carlo variant, the target is computed from the complete return until the end of the episode:

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k}.$$

In this case, the actor can be updated using the full return or an advantage estimate based on the full return:

$$A_t = G_t - V_w(s_t).$$

Monte Carlo updates do not use bootstrapping from the next-state value estimate, which removes bootstrap bias. However, they usually have higher variance and require waiting until the episode is finished before performing the update. This idea is closely related to Monte Carlo policy-gradient methods such as REINFORCE [9], [13].

The actor and critic are therefore updated together. The actor improves the policy based on feedback from the critic, while the critic improves its value estimates using rewards observed from the environment. Compared with pure policy-gradient methods, Actor-Critic methods usually reduce variance by using a learned value function as a baseline, while still directly optimizing a parameterized policy [3], [9].

### III. IMPLEMENTATION

The project was implemented in Python, using OpenCV for frame processing and object detection, and PyTorch for neural network implementation and training [7], [8]. DOSBox is used to run the original *Donkey* game in a controlled environment suitable for interaction with the learning agents [2].

This section describes the main implementation components, including the environment setup, object detection, state vector representation, reward system, and the implementation of the reinforcement learning agents.

#### A. Environment Setup

The environment uses DOSBox, an x86 DOS emulator, to run the original *Donkey* game on modern systems [2]. A Python script launches the emulator, loads the configuration, and starts the game, allowing the training pipeline to control initialization and interaction with the environment.

#### B. Object Detection

The main objects in the game screen are detected using OpenCV template matching. This method searches for a small template image inside a larger frame by comparing the template with possible image regions using `cv.matchTemplate()` [7].

In this project, separate templates are used for detecting the car and the obstacle. For each captured frame, the best matching position is used to determine the coordinates of the corresponding object.



(a) car template



(b) obstacle template

Fig. 3. Templates used for detecting the main objects in the *Donkey* game: (a) car template and (b) obstacle template.

#### C. State Vector Representation

The agent uses a compact state vector instead of raw pixels. The state is created from the detected positions of the car and the donkey:

$$s_t = [\text{car\_line}, \text{donkey\_line}, \text{danger}].$$

The variables `car_line` and `donkey_line` describe the lanes of the car and the donkey. The variable `danger` indicates whether the donkey is close enough to require a reaction. Therefore, the state size is:

$$|s_t| = 3.$$

The action space contains two actions: 0 for no jump and 1 for jump.

#### D. Episode Definition

An episode represents one game attempt. Before the episode starts, the environment waits until the player is detected on the screen. After that, the agent repeatedly observes the current state, selects an action, executes it in the game, and receives a reward based on the next observed screen state.

The episode continues until the agent crashes into the donkey. In the implementation, a crash is detected when the stable donkey score counter increases. This event is treated as the terminal state of the episode.

During each step, the transition is passed to the selected learning agent. For one-step methods, training can be performed after each transition. For episodic methods, transitions are stored and used after the episode ends.

#### E. Reward System

The reward system is based on two main goals: avoiding crashes and increasing the driver score. A crash is the worst possible event, because it gives a point to the donkey and ends the episode. Therefore, it receives a reward of  $-100$ . If the agent completes a lap and increases the driver score, it receives a reward of  $+100$ . Otherwise, the agent receives a step reward of  $+1$ , which encourages longer survival.

The base reward is defined as:

$$R_{\text{base}} = \begin{cases} -100, & \text{if the agent crashes,} \\ 100 + 1, & \text{if a lap is completed,} \\ 1, & \text{otherwise, as a step reward.} \end{cases}$$

Additional reward shaping is used to teach the agent when jumping is useful. The agent is penalized for unnecessary jumps and for missing dangerous situations. A small positive reward is given when the agent jumps while danger is detected:

$$R_{\text{shape}} = \begin{cases} -12, & \text{jump, donkey visible, different lane,} \\ -10, & \text{jump, no danger,} \\ -8, & \text{no jump, danger,} \\ +3, & \text{jump, danger} \end{cases}$$

The final reward is computed as:

$$R_t = \begin{cases} R_{\text{base}}, & \text{if the episode is terminal,} \\ R_{\text{base}} + R_{\text{shape}}, & \text{otherwise.} \end{cases}$$

#### F. Implemented Agents

1) *Q-learning Agent*: The Q-learning agent was implemented as a Deep Q-Network (DQN), where the Q-function is approximated by a neural network with parameters  $\theta$ . The network receives the state vector as input and outputs Q-values for each possible action. A separate target network with parameters  $\theta^-$  is maintained and updated periodically every 500 training steps to stabilize the learning process.

Experience replay is used to reduce correlations between consecutive samples. Transitions  $(s_t, a_t, r_t, s_{t+1}, d_t)$  are stored

TABLE I  
HYPERPARAMETERS USED FOR THE DQN AND DOUBLE DQN AGENTS.

| Hyperparameter           | Value     | Description and usage                                                        |
|--------------------------|-----------|------------------------------------------------------------------------------|
| Discount factor $\gamma$ | 0.99      | Defines the importance of future rewards. Used in the TD target calculation. |
| Learning rate            | 0.0001    | Controls the update size of the network parameters.                          |
| $\epsilon$ initial       | 1.0       | Initial exploration probability.                                             |
| $\epsilon$ minimum       | 0.05      | Minimum exploration probability after decay.                                 |
| $\epsilon$ decay         | 0.9995    | Multiplicative decay factor applied after each training step.                |
| Target network update    | 500 steps | Frequency of copying training network weights to the target network.         |
| Reward scale             | 100.0     | Scales raw rewards before target calculation.                                |
| Maximum gradient norm    | 1.0       | Limits gradient size during optimization.                                    |

in a replay buffer and sampled randomly during training. The loss is computed using the Huber loss:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{Huber}} \left( Q_{\theta}(s_t, a_t), r_t + \gamma \max_{a'} Q_{\theta^-}(s_{t+1}, a') \right).$$

Action selection follows an  $\epsilon$ -greedy policy, where  $\epsilon$  decays exponentially from 1.0 to 0.05 during training.

2) *Double Q-learning Agent*: The Double Q-learning agent shares the same architecture and hyperparameters as the Q-learning agent. The only difference is in the target value computation. Instead of using the target network for both action selection and evaluation, the training network selects the action and the target network evaluates it:

$$y_t = r_t + \gamma Q_{\theta^-} \left( s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a') \right).$$

This modification reduces the overestimation bias present in standard Q-learning without requiring any additional network or significant computational overhead.

3) *Actor-Critic Agent*: Based on the Actor-Critic method described in the previous section, two Actor-Critic variants were implemented. The first variant is a one-step Actor-Critic agent, which updates the actor and critic after each transition using a temporal-difference target. The second variant is an episodic Actor-Critic agent, which updates the networks after the episode ends using Monte Carlo returns.

Both variants use a separate actor network and critic network. The actor represents the policy  $\pi_{\theta}(a|s)$ , while the critic estimates the state-value function  $V_w(s)$ .

## IV. EXPERIMENTAL EVALUATION

This section describes the training and evaluation setup used for comparing the implemented reinforcement learning agents. The experiments were performed using multiple random seeds and different neural network configurations in order to evaluate the stability of the learning process.

#### A. Training Procedure

Each agent was trained in the Donkey environment for up to 20000 episodes. An episode ended when the agent crashed into the donkey or reached the maximum episode length:

$$\text{MAX\_EPISODE\_STEPS} = 500.$$

TABLE II  
HYPERPARAMETERS USED FOR THE ACTOR-CRITIC AGENTS.

| Hyperparameter              | Value   | Description and usage                                                                               |
|-----------------------------|---------|-----------------------------------------------------------------------------------------------------|
| Discount factor $\gamma$    | 0.97    | Defines the importance of future rewards. Used in the TD target and Monte Carlo return calculation. |
| Actor learning rate         | 0.0003  | Controls the update size of the actor network parameters.                                           |
| Critic learning rate        | 0.0003  | Controls the update size of the critic network parameters.                                          |
| Entropy coefficient $\beta$ | 0.01    | Controls the strength of the entropy bonus used to support exploration.                             |
| Reward scale                | 100.0   | Scales raw rewards before target or return calculation.                                             |
| Maximum gradient norm       | 1.0     | Limits gradient size during optimization.                                                           |
| Return normalization        | enabled | Normalizes Monte Carlo returns in the episodic Actor-Critic variant.                                |

Reaching this limit was treated as a successful long-survival episode. To evaluate training stability, the agents were trained with four random seeds:

$$\{64, 164, 264\}.$$

For the seed-based experiment, the hidden layer configuration was fixed to:

$$[64, 64].$$

To evaluate the influence of network size, the following hidden layer configurations were tested:

$$[64], \quad [64, 64], \quad [64, 64, 64].$$

For this architecture-based experiment, the random seed was fixed to 164.

### B. Q-learning Results

The total training time for each independent run is summarized in Table III. The summary statistics across all runs are shown in Table IV. The learning performance is shown in

TABLE III  
TRAINING TIME OF THE Q-LEARNING AGENT FOR DIFFERENT RANDOM SEEDS.

| Seed | Episodes | Training time [min] |
|------|----------|---------------------|
| 64   | 246      | 58.40               |
| 164  | 190      | 24.30               |
| 264  | 190      | 29.77               |

TABLE IV  
SUMMARY STATISTICS OF THE Q-LEARNING AGENT ACROSS RANDOM SEEDS.

|      | Episodes | Training time [min] | Mean reward |
|------|----------|---------------------|-------------|
| Mean | 208.67   | 37.49               | 341.37      |
| Std  | 26.35    | 15.60               | 516.55      |

Figure 4, which presents the mean reward curve together with the reward variance across all runs, providing a view of both the overall learning trend and the stability of the agent.

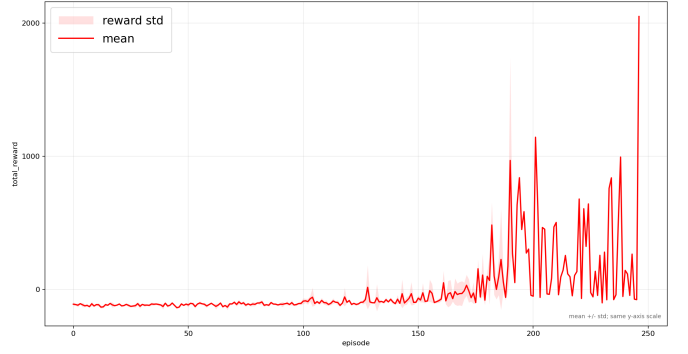


Fig. 4. Mean total reward per episode with variance across different random seeds for the Q-learning agent.

1) *Influence of Hidden Layer Configuration:* The results for different hidden layer configurations are summarized in Table V. The corresponding reward curves are shown in Figure 5.

TABLE V  
INFLUENCE OF HIDDEN LAYER CONFIGURATION ON Q-LEARNING TRAINING.

| Hidden layers  | Episodes | Time [min] |
|----------------|----------|------------|
| [ 64 ]         | 203      | 34.52      |
| [ 64, 64 ]     | 190      | 24.30      |
| [ 64, 64, 64 ] | 179      | 28.24      |

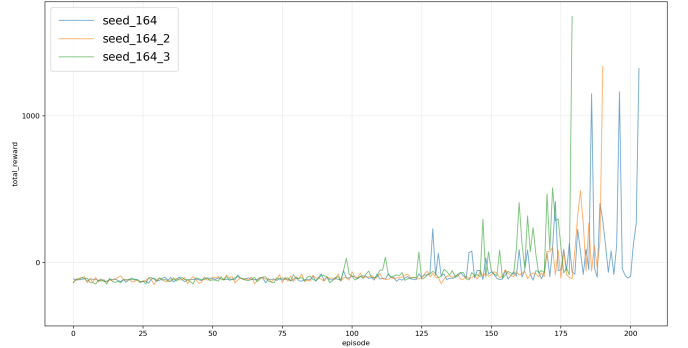


Fig. 5. Total reward per episode for different hidden layer configurations of the Q-learning agent.

### C. Double Q-learning Results

The total training time for each independent run is summarized in Table VI. The summary statistics across all runs are shown in Table VII. The learning performance is shown in

TABLE VI  
TRAINING TIME OF THE DOUBLE Q-LEARNING AGENT FOR DIFFERENT RANDOM SEEDS.

| Seed | Episodes | Training time [min] |
|------|----------|---------------------|
| 64   | 233      | 55.91               |
| 164  | 187      | 30.62               |
| 264  | 187      | 32.81               |

TABLE VII  
SUMMARY STATISTICS OF THE DOUBLE Q-LEARNING AGENT ACROSS  
RANDOM SEEDS.

|      | Episodes | Training time [min] | Mean reward |
|------|----------|---------------------|-------------|
| Mean | 202.33   | 39.78               | 283.23      |
| Std  | 21.59    | 11.49               | 407.99      |

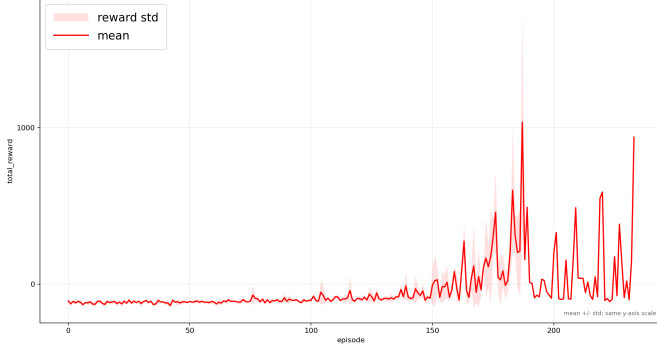


Fig. 6. Mean total reward per episode with variance across different random seeds for the Double Q-learning agent.

Figure 6, which presents the mean reward curve together with the reward variance across all runs, providing a view of both the overall learning trend and the stability of the agent.

1) *Influence of Hidden Layer Configuration:* The results for different hidden layer configurations are summarized in Table VIII. The corresponding reward curves are shown in Figure 7.

TABLE VIII  
INFLUENCE OF HIDDEN LAYER CONFIGURATION ON DOUBLE  
Q-LEARNING TRAINING.

| Hidden layers | Episodes | Time [min] |
|---------------|----------|------------|
| [64]          | 187      | 32.89      |
| [64, 64]      | 187      | 30.62      |
| [64, 64, 64]  | 348      | 98.28      |

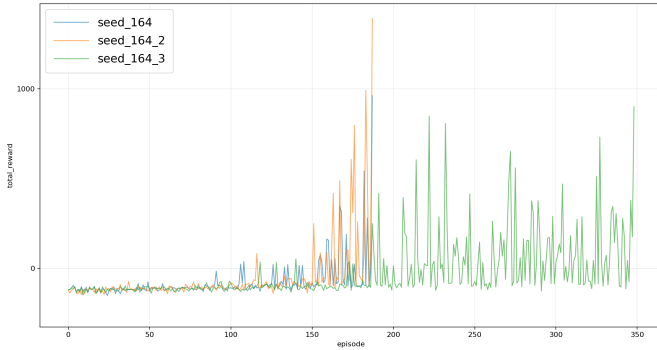


Fig. 7. Total reward per episode for different hidden layer configurations of the Double Q-learning agent.

#### D. One-step Actor-Critic Results

The total training time for each independent run is summarized in Table IX.

TABLE IX  
TRAINING TIME OF THE ONE-STEP ACTOR-CRITIC AGENT FOR DIFFERENT  
RANDOM SEEDS.

| Seed | Episodes | Time [min] | Mean reward | Reward std |
|------|----------|------------|-------------|------------|
| 64   | 134      | 19.63      | -67.56      | 252.14     |
| 164  | 120      | 15.83      | -79.07      | 206.64     |
| 264  | 143      | 23.21      | -60.73      | 190.83     |

The learning performance is shown in Figures 8 and 9. The first figure shows the total reward per episode for each independent run. The second figure shows the mean reward curve together with the reward variance across all runs.

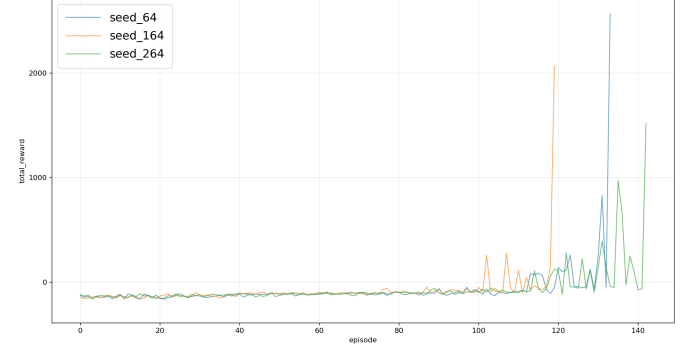


Fig. 8. Total reward per episode for the one-step Actor-Critic agent across different random seeds.

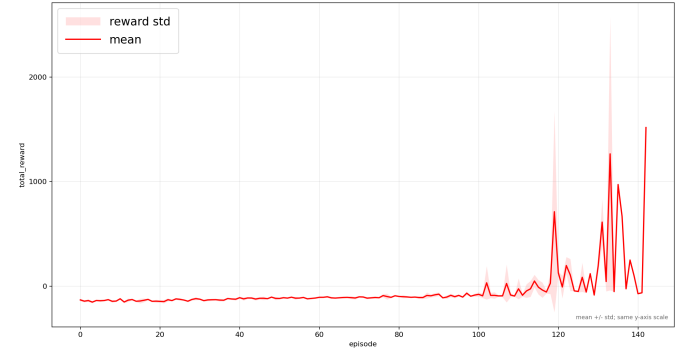


Fig. 9. Mean total reward per episode with variance across different random seeds.

The first graph shows the variability between individual training runs. The second graph provides a clearer view of the overall learning trend and the spread of the achieved rewards. Together, these figures make it possible to evaluate both the average performance and the stability of the one-step Actor-Critic agent.

1) *Influence of Hidden Layer Configuration:* \* The results for different hidden layer configurations are summarized in Table X. The corresponding reward and temporal-difference error curves are shown in Figures 10 and 11.

The reward curves show that all tested configurations were able to produce occasional high-reward episodes, but the learning process remained unstable. The [64] configuration

TABLE X  
INFLUENCE OF HIDDEN LAYER CONFIGURATION ON ONE-STEP  
ACTOR-CRITIC TRAINING.

| Hidden layers | Episodes | Time [min] | Mean reward | Reward std |
|---------------|----------|------------|-------------|------------|
| [64]          | 302      | 38.19      | -78.44      | 148.93     |
| [64, 64]      | 120      | 15.83      | -79.07      | 206.64     |
| [64, 64, 64]  | 57       | 8.59       | -70.19      | 222.32     |

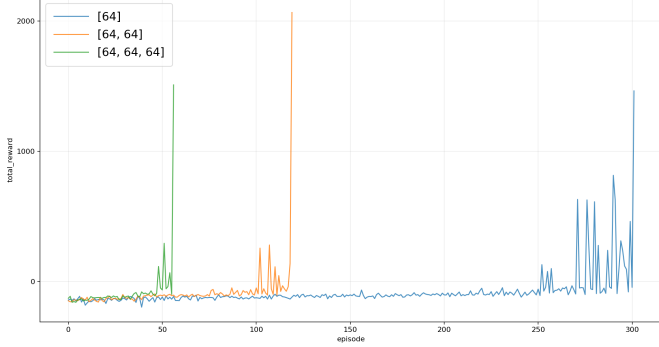


Fig. 10. Total reward per episode for different hidden layer configurations of the one-step Actor-Critic agent.

trained for the longest number of episodes and showed later reward improvements, while the deeper configurations produced shorter training runs with isolated reward spikes. Therefore, increasing the number of hidden layers did not lead to a clearly more stable reward curve.

The temporal-difference error decreased for all configurations, which indicates that the critic became more consistent during training.

#### E. Episodic Actor-Critic Results

The episodic Actor-Critic agent was evaluated under the following training conditions:

- 1) **NUMBER\_OF\_SEED:** 164;
- 2) **HIDDEN\_LAYERS\_SIZE:** [64, 64].

The total training time for this experiment was **96.36 minutes**. The main results are shown in Figures 12 and 13.

The episode length does not show a stable improvement during training. Although several longer episodes appear, they are isolated and are not followed by consistent progress. Most episodes remain short, which indicates that the learned policy was unreliable.

The advantage values also remain highly unstable. Large fluctuations suggest that the learning signal provided to the actor was inconsistent. As a result, the policy updates did not lead to stable improvement.

A likely reason is the **reward structure**. Longer episodes could accumulate more penalties for unnecessary or incorrect jumps. As a result, a **longer episode could receive a worse return than a shorter episode** that ended earlier. This made the *learning signal unstable* and reduced the effectiveness of the episodic Actor-Critic variant.

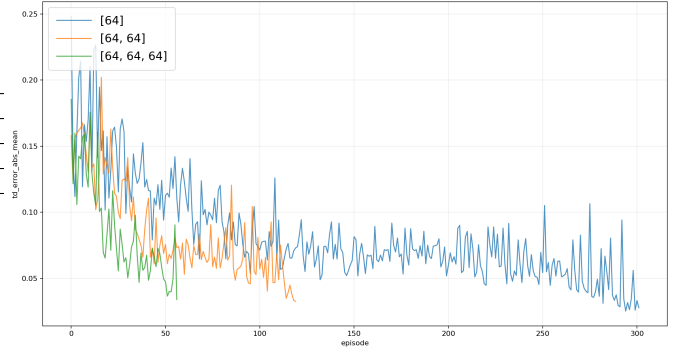


Fig. 11. Temporal-difference error for different hidden layer configurations of the one-step Actor-Critic agent.

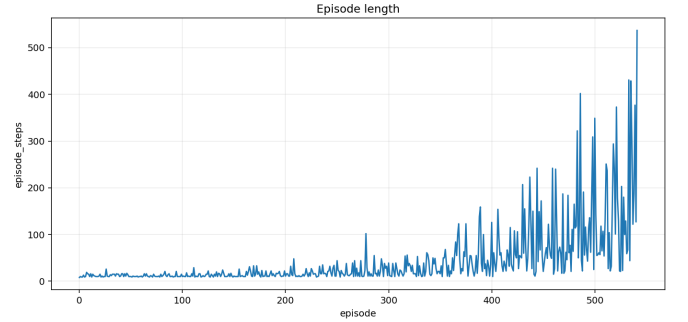


Fig. 12. Episode length for the episodic Actor-Critic agent.

## V. CONCLUSION

This work presented the implementation and comparison of three reinforcement learning approaches for controlling an agent in the classic MS-DOS game *Donkey*: Q-learning, Double Q-learning, and an Actor-Critic method.

The implemented agents showed that reinforcement learning can be used to learn obstacle-avoidance behaviour in the *Donkey* environment. Both the Q-learning and Double Q-learning agents converged within a comparable number of episodes and showed a clear upward trend in episode length during training. The one-step Actor-Critic agent also showed learning progress, while the episodic Actor-Critic variant struggled due to the reward structure of the environment, where longer episodes could accumulate more penalties and produce a worse return than shorter ones.

Regarding the influence of hidden layer configuration, the results showed that the effect of network depth depended on the selected algorithm. For the Q-learning agent, all three configurations converged in a similar number of episodes and time. For the Double Q-learning agent, the [64, 64, 64] configuration required significantly more episodes (348 vs. 187) and nearly three times more training time (98.28 vs. 30.62 minutes) without achieving better results. Therefore, increasing network depth was not beneficial for the value-based agents in this environment, where a simpler architecture was already sufficient.



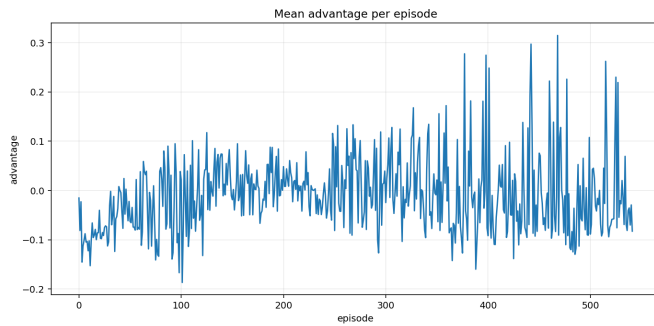


Fig. 13. Advantage values for the episodic Actor-Critic agent.

For the Actor-Critic method, the influence of hidden layers was different. The three-hidden-layer configuration produced better results than the two-layer variant in the tested setup. This suggests that the Actor-Critic agent benefited from additional network capacity. However, the training process was still not fully stable. The temporal-difference error and loss-related metrics generally moved toward convergence, but they also showed noticeable fluctuations. This indicates that the value estimation improved during training, but the learning signal remained unstable.

Overall, the [64, 64] configuration provided the best balance between training time and achieved performance for the value-based methods. For the Actor-Critic method, the three-hidden-layer configuration produced the best result among the tested configurations. Future work should focus on stabilizing the agents by improving the reward function, refining the state representation, testing additional network architectures, and tuning hyperparameters for each algorithm.

## REFERENCES

- [1] DONKEY.BAS. <https://en.wikipedia.org/wiki/DONKEY.BAS>. Accessed: 2026-05-05.
- [2] DOSBox Team. DOSBox, an x86 emulator with DOS. <https://www.dosbox.com/>. Accessed: 2026-05-05.
- [3] Vijay R. Konda and John N. Tsitsiklis. Actor-critic algorithms. In *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999.
- [4] Nicholas Metropolis and Stanislaw Ulam. The monte carlo method. *Journal of the American Statistical Association*, 44(247):335–341, 1949.
- [5] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1928–1937. PMLR, 2016.
- [6] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [7] OpenCV. Template Matching. [https://docs.opencv.org/4.x/d4/dc6/tutorial\\_py\\_template\\_matching.html](https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html). Accessed: 2026-05-05.
- [8] PyTorch. PyTorch Documentation. <https://docs.pytorch.org/>. Accessed: 2026-05-05.
- [9] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [10] Hado van Hasselt. Double q-learning. In *Advances in Neural Information Processing Systems*, 2010.
- [11] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, pages 2094–2100, 2016.
- [12] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [13] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- [14] Henry Wu. Intro to reinforcement learning: Monte carlo to policy gradient. <https://medium.com/@hsinhungw/intro-to-reinforcement-learning-monte-carlo-to-policy-gradient-1c7ede4eed6e>, 2024. Accessed: 2026-05-05.